

Mapping A Python Class to a Database Table : Code-Along (CodeGrade)

Learning Goals

- Persist the attributes of a Python object as a row in a database table.
-

Key Vocab

- **Object-Relational Mapping (ORM)**: a programming technique that provides a mapping between an object-oriented data model and a relational database model.
-

Introduction

Object-Relational Mapping (ORM) is a programming technique that provides a mapping between an object-oriented data model and a relational database model.

We equate a Python **class** with a database **table** and an instance of a class (i.e. an object) to a **table row**.

Python	Relational Database
Class	Table
Object	Row

Why map classes to tables? To persist data stored in Python objects efficiently and in an organized manner, we need to map a Python class to a database table by writing methods that encapsulate table creation and deletion, along with methods to save, update, delete, and query object state within a database table.

As an example, assume we want to create a database to store data about the departments and employees within a company. It is convention to pluralize the name of the class to create the name of the table. Therefore, the `Department` class maps to the "departments" table and the `Employee` class maps to the "employees" table.

Python Class	Relational Database Table
Department	departments
Employee	employees

In this lesson, we will learn how to persist a Python object as a row in a database table by implementing the following methods for a class:

Method	Return	Description
<code>create_table(cls)</code>	None	Create a table to store data about instances of a class.
<code>drop_table(cls)</code>	None	Drop the table.
<code>save(self)</code>	None	Save the attributes of an object as a new table row.
<code>create(cls, attributes)</code>	an object that is an instance of <code>cls</code>	Create a new object that is an instance of <code>cls</code> and save its attributes as a new table row.
<code>update(self)</code>	None	Update an object's corresponding table row
<code>delete(self)</code>	None	Delete the table row for the specified object

This lesson explains how to map a single class to a database table. Techniques for mapping relationships between multiple classes will be covered in a separate lesson.

Mapping a `Department` class to a database table

This lesson is a code-along, so fork and clone the repo.

NOTE: Remember to run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
pipenv install
pipenv shell
```

The starter code for the `Department` class is in `lib/department.py`. The `Department` class is defined with attributes for `id`, `name` and `location`.

The `__init__` method assigns a default value of `None` to the `id` attribute. The `id` will be assigned a value *after* saving the object attributes as a new table row (`id` will be assigned the value of the new row's primary key). We'll see how to assign the `id` attribute later in the lesson.

```
from __init__ import CURSOR, CONN
```

```
class Department:
```

```
    def __init__(self, name, location, id=None):
        self.id = id
        self.name = name
        self.location = location
```

```
    def __repr__(self):
        return f"<Department {self.id}: {self.name}, {self.location}>"
```

Creating the Database

First we need to create our database, then we will create a "departments" table.

Whose responsibility is it to create the database? It is not the responsibility of the `Department` class. Remember, classes are mapped to *tables inside a database*, not to the database as a whole. Accordingly, you'll see that Python packages have modules solely for configuration of reused (constant) variables. We'll put the database initialization in the file `lib/__init__.py`.

```
# lib/__init__.py
import sqlite3

CONN = sqlite3.connect('company.db')
CURSOR = CONN.cursor()
```

- `CONN` is a constant equal to a hash that contains a connection to the database.
- `CURSOR` is a constant that allows us to interact with the database and execute SQL statements.

We can access the constants within `lib/department.py` by adding the import statement before the class declaration:

```
from __init__ import CURSOR, CONN

class Department:

    # ... existing class attributes and methods
```

The starter code for these files is set up, so you can explore it and code along with the rest of this lesson.

► Which constant will we use to execute SQL statements: `CONN` or `CURSOR` ?

Creating (and dropping) the "departments" table

Method	Return	Description
<code>create_table (cls)</code>	None	Create a table to store data about instances of a class.
<code>drop_table (cls)</code>	None	Drop the table.

To "map" our `Department` class to a database table, we will:

- create a table with the plural of the class name, i.e. "departments"
- assign column names that match the instance attributes of the class, i.e. `id` , `name` , and `location`

Update the `Department` class to add the `create_table` and `drop_table` methods after the existing methods:

```
from __init__ import CURSOR, CONN

class Department:

    # add new ORM methods after existing methods ....

    @classmethod
    def create_table(cls):
        """ Create a new table to persist the attributes of Department instances """
        sql = """
            CREATE TABLE IF NOT EXISTS departments (
                id INTEGER PRIMARY KEY,
                name TEXT,
                location TEXT)
            """
        CURSOR.execute(sql)
        CONN.commit()

    @classmethod
    def drop_table(cls):
        """ Drop the table that persists Department instances """
        sql = """
            DROP TABLE IF EXISTS departments;
            """
        CURSOR.execute(sql)
        CONN.commit()
```

Why are the `create_table()` and `drop_table()` methods class methods? Well, it is *not* the responsibility of an individual department object to create the table it will eventually be saved into, it is the job of the class as a whole.

You can try out this code now to create the "departments" table in the `company.db` database file. Check out the code in the `debug.py` file:

```
#!/usr/bin/env python3

from __init__ import CONN, CURSOR
from department import Department

import ipdb
ipdb.set_trace()
```

In this file, we're importing in the `sqlite3.Connection` and `sqlite3.Cursor` objects that we instantiated in `lib/__init__.py`. We're also importing the `Department` class so that we can use its methods during our `ipdb` session.

Run `python debug.py` to enter the `ipdb` session:

```
python lib/debug.py
```

then run the `create_table()` class method:

```
ipdb> Department.create_table()
```

Creating a table doesn't return any data, so SQLite returns `None`. If you'd like to confirm that the table was created successfully, you can run a special `PRAGMA` command to show the information about the `departments` table:

```
ipdb> CURSOR.execute("PRAGMA table_info(departments)").fetchall()
# => [(0, 'id', 'INTEGER', 0, None, 1), (1, 'name', 'TEXT', 0, None, 0), (2, 'location', 'TEXT', 0, None, 0)]
```

The output isn't easy to read, but you'll see the column names (`id` , `name` , `location`) along with their data types (`INTEGER` , `TEXT` , `TEXT`).

We can also use the [VS Code SQLITE Explorer extension](https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite)  (<https://marketplace.visualstudio.com/items?itemName=alexcvzz.vscode-sqlite>) to view the database and table. Right-click on the `company.db` file (located in the `lib` folder), then select "Open Database":

Expanding the `SQLITE EXPLORER` menu item lets us see the database schema:

Another option is to use the [VS Code SQLITE Viewer extension](https://marketplace.visualstudio.com/items?itemName=qwtel.sqlite-viewer)  (<https://marketplace.visualstudio.com/items?itemName=qwtel.sqlite-viewer>) to view the database tables. After installing the extension, right-click on the `company.db` file then select "Open with..." and then choose "SQLite Viewer".

If we want to delete the table from the database, we would execute the following in `ipdb` :

```
ipdb> Department.drop_table()
```

Confirm the table is deleted using the `SQLITE EXPLORER` (or retype the `PRAGMA` command in the `ipdb` session).

We'll be making lots of changes to our `Department` class to experiment with different ways to persist the data. Let's update `debug.py` to drop then recreate the table, so we always start with a fresh table with no data.

Make sure to first exit out of `ipdb` by typing `exit()` or by pressing `ctrl-d` . Then update `debug.py` as shown:

```
#!/usr/bin/env python3

from __init__ import CONN, CURSOR
from department import Department

import ipdb
```

```
Department.drop_table()
Department.create_table()

ipdb.set_trace()
```

Drop and recreate the table by executing the updated `debug.py` file:

```
python lib/debug.py
```

Confirm the table has been recreated using either the SQLITE EXPLORER extension, or by executing the PRAGMA statement in the `ipdb` session.

Mapping An Object to A Table Row

Method	Return	Description
save (self)	None	Save the attributes of an object as a new table row.

Now that we have the database and "departments" table, we can start persisting object data as rows in the table. Note, **we are not saving Python objects in our database**. Rather, we are going to take the individual attributes of a given object, in this case a department's `name` and `location`, and save those attributes to the database as a single row. The row will also include a primary key column named `id`.

For example, persisting the `name` and `location` attributes of two different instances of the `Department` class might result in a "departments" table that looks like this:

We'll persist an object that is an instance of the `Department` class as a row in a "departments" table with a new **instance method** named `save()`.

The overall process to save the attributes of a specific `Department` object to the database is:

- Insert a new row into the "departments" table that contains the attribute values of the object.

- Grab the primary key `id` column of that newly inserted row and assign that value as the `id` attribute of the object.

Add the `save(self)` method to the end of the `Department` class:

```
from __init__ import CURSOR, CONN

class Department:

    # existing methods ...

    def save(self):
        """ Insert a new row with the name and location values of the current Department instance.
        Update object id attribute using the primary key value of new row.
        """
        sql = """
            INSERT INTO departments (name, location)
            VALUES (?, ?)
        """

        CURSOR.execute(sql, (self.name, self.location))
        CONN.commit()

        self.id = CURSOR.lastrowid
```

Notice the insert statement contains two question marks rather than string literals for the `name` and `location` values. We need to pass in, or interpolate, the name and location of a given `Department` object into our Python string that represents the SQL insert statement.

We use something called **bound parameters** to achieve this.

Important: using f-strings or the `str.format()` method will not work with statements sent through the `sqlite3` module. `sqlite3` will interpret any values interpolated in this fashion as *columns*. Weird!

Bound Parameters

Bound parameters protect our program from getting confused by [SQL injections](https://en.wikipedia.org/wiki/SQL_injection) and special characters. Instead of interpolating variables into a Python string containing SQL syntax, we use the `?` characters as placeholders. Then, the special magic provided to us by the `sqlite3` module's `Cursor.execute()` method will take the values we pass in as an argument tuple `(self.name, self.location)` and apply them as the values of the question marks.

We can step through this process by instantiating and saving objects that are instances of the `Department` class, printing the object state before and after saving to the database. Update `debug.py` as shown below, then execute `python lib/debug.py` to see the result of each print statement (make sure to exit out of `ipdb` with `exit()` or `ctrl+D` in order to reload the code if you left it open earlier).

```
#!/usr/bin/env python3
```

```
from __init__ import CONN, CURSOR
from department import Department
```

```
import ipdb
```

```
Department.drop_table()
Department.create_table()
```

```
payroll = Department("Payroll", "Building A, 5th Floor")
print(payroll) # <Department None: Payroll, Building A, 5th Floor>
```

```
payroll.save() # Persist to db, assign object id attribute
print(payroll) # <Department 1: Payroll, Building A, 5th Floor>
```

```
hr = Department("Human Resources", "Building C, East Wing")
print(hr) # <Department None: Human Resources, Building C, East Wing>
```

```
hr.save() # Persist to db, assign object id attribute
print(hr) # <Department 2: Human Resources, Building C, East Wing>

ipdb.set_trace()
```

- Prior to calling the `save()` method, the print statement shows the newly instantiated `Department` object's `id` attribute initially has the value of `None`.
- After the `save` method is executed, the print statement shows the `Department` object's `id` attribute has been updated to contain an integer value corresponding to the primary key of the new table row.

The `save()` method does not return a value, but we can query the database table and create a list from the result.

Execute this code by entering **one statement at a time** to the `ipdb>` prompt:

```
ipdb> departments = CURSOR.execute('SELECT * FROM departments')
ipdb> [row for row in departments]
# => [(1, 'Payroll', 'Building A, 5th Floor'), (2, 'Human Resources', 'Building C, East Wing')]
```

We can also use VS Code's SQLITE Explorer to show the table results. Click the "Show Table" arrow to view the table rows:

Creating Instances vs. Creating Table Rows

The moment in which we create a new object that is an instance of the `Department` class with the `__init__` method is *different than the moment in which we save a representation of that department object to our database*.

- The `__init__` method creates a new Python object, an instance of the `Department` class.
- The `save()` method takes the attributes that characterize the Python object and saves them in a new row in the database "departments" table.

While it is possible to update the `__init__` method to immediately save the object's attributes as a new table row, this is not a great idea. We don't want to force our objects to be saved every time they are created, or make the creation of an object dependent upon/always coupled with saving a row to the database. So, we'll keep our `__init__` and `save()` methods separate, allowing the programmer to decide when each method should be called.

The `create()` Method

Method	Return	Description
<code>create(cls, attributes)</code>	an object that is an instance of <code>cls</code>	Create a new object that is an instance of <code>cls</code> and save its attributes as a new table row.

The `save()` method requires two steps to persist an object to the database:

1. Create an object that is an instance of the `Department` class, then
2. Call the `save()` method to insert a new row containing the object's attribute values to the database.

Let's define a new class method named `create()` that does just that in one step. We use a class method because our object does not exist at the time the method is called.

```
class Department:

    # ... existing methods

    @classmethod
    def create(cls, name, location):
        """ Initialize a new Department instance and save the object to the database """
        department = cls(name, location)
        department.save()
        return department
```

Here, we use arguments to pass a name and location into our `create()` method. We use that name and location to instantiate an object that is a new instance of the `Department` class. Then, we call the `save()` method to persist the new object's attributes to the database.

Notice that at the end of the method, we are returning the `Department` object that we instantiated.

Edit `debug.py` and let's use the `create()` method to instantiate and save the payroll and human resources departments:

```
#!/usr/bin/env python3

from __init__ import CONN, CURSOR
from department import Department

import ipdb

Department.drop_table()
Department.create_table()

payroll = Department.create("Payroll", "Building A, 5th Floor")
print(payroll) # <Department 1: Payroll, Building A, 5th Floor>

hr = Department.create("Human Resources", "Building C, East Wing")
print(hr) # <Department 2: Human Resources, Building C, East Wing>
```

Run the file using `python lib/debug.py`, then try querying the table in the `ipdb` session or use the SQLITE EXPLORER extension to confirm the new table row for the accounting department.

Update and delete methods

Method	Return	Description
<code>update(self)</code>	None	Update an object's corresponding table row
<code>delete (self)</code>	None	Delete the table row for the specified object

Edit the `Department` class to add methods to update and delete the database row associated an object that is an instance of the `Department` class:

```

def update(self):
    """Update the table row corresponding to the current Department instance."""
    sql = """
        UPDATE departments
        SET name = ?, location = ?
        WHERE id = ?
    """
    CURSOR.execute(sql, (self.name, self.location, self.id))
    CONN.commit()

def delete(self):
    """Delete the table row corresponding to the current Department instance"""
    sql = """
        DELETE FROM departments
        WHERE id = ?
    """

    CURSOR.execute(sql, (self.id,))
    CONN.commit()

```

Once again we use bound parameters, with each question mark `?` bound to a value within the `CURSOR.execute` method call.

Let's edit `debug.py` to call the new `update()` and `delete()` methods:

```

#!/usr/bin/env python3

from __init__ import CONN, CURSOR
from department import Department

import ipdb

Department.drop_table()

```

```
Department.create_table()

payroll = Department.create("Payroll", "Building A, 5th Floor")
print(payroll) # <Department 1: Payroll, Building A, 5th Floor>

hr = Department.create("Human Resources", "Building C, East Wing")
print(hr) # <Department 2: Human Resources, Building C, East Wing>

hr.name = 'HR'
hr.location = "Building F, 10th Floor"
hr.update()
print(hr) # <Department 2: HR, Building F, 10th Floor>

print("Delete Payroll")
payroll.delete() # delete from db table, object still exists in memory
print(payroll) # <Department 1: Payroll, Building A, 5th Floor>

ipdb.set_trace()
```

Run `python lib/debug.py` .

You can use the SQLITE EXPLORER extension to confirm the table contents, or use `ipdb` to query the table and confirm the updated/deleted table rows. Enter the following statements one at a time at the `ipdb>` prompt.

```
ipdb> departments = CURSOR.execute('SELECT * FROM departments')
ipdb> [row for row in departments]
# => [(2, 'HR', 'Building F, 10th Floor')]
```

Try to use the `ipdb` session to experiment with creating/creating/deleting additional `Department` objects in the database.

Testing the ORM

The `testing` folder contains a file `department_orm_test.py` that tests the ORM methods.

Run `pytest -x` to confirm your code passes the tests, then submit this code-along assignment using `git`.

Note: You may have to delete your existing database `company.db` for all of the tests to pass- SQLite sometimes "locks" databases that have been accessed by multiple modules.

Conclusion

We've seen how to map a Python **class** with a database **table** and an **instance of a class (i.e. object)** to a **table row**.

Python	Relational Database
Class	Table
Object	Row

The important concept to grasp here is the idea that we are *not* saving Python objects into our database. We are saving the attribute values for a Python object as a new row in our database table. Our `departments` table contains one column for each instance attribute defined in the `Department` class.

An instance of a class resides in a part of the program's runtime memory called the heap, while table row data is stored in a database file. Creating, updating, or deleting a Python object that is stored in the heap will not affect what is stored in the database file, unless we explicitly call an ORM method to keep the separate memory spaces in sync.

Solution Code

```
from __init__ import CURSOR, CONN

class Department:

    def __init__(self, name, location, id=None):
        self.id = id
```



```
self.name = name
self.location = location

def __repr__(self):
    return f"<Department {self.id}: {self.name}, {self.location}>"

@classmethod
def create_table(cls):
    """ Create a new table to persist the attributes of Department instances """
    sql = """
        CREATE TABLE IF NOT EXISTS departments (
            id INTEGER PRIMARY KEY,
            name TEXT,
            location TEXT)
        """
    CURSOR.execute(sql)
    CONN.commit()

@classmethod
def drop_table(cls):
    """ Drop the table that persists Department instances """
    sql = """
        DROP TABLE IF EXISTS departments;
        """
    CURSOR.execute(sql)
    CONN.commit()

def save(self):
    """ Insert a new row with the name and location values of the current Department instance.
    Update object id attribute using the primary key value of new row.
    """
    sql = """
        INSERT INTO departments (name, location)
```

```

        VALUES (?, ?)
    """

    CURSOR.execute(sql, (self.name, self.location))
    CONN.commit()

    self.id = CURSOR.lastrowid

    @classmethod
    def create(cls, name, location):
        """ Initialize a new Department instance and save the object to the database """
        department = cls(name, location)
        department.save()
        return department

    def update(self):
        """Update the table row corresponding to the current Department instance."""
        sql = """
            UPDATE departments
            SET name = ?, location = ?
            WHERE id = ?
        """
        CURSOR.execute(sql, (self.name, self.location, self.id))
        CONN.commit()

    def delete(self):
        """Delete the table row corresponding to the current Department instance"""
        sql = """
            DELETE FROM departments
            WHERE id = ?
        """

        CURSOR.execute(sql, (self.id,))

```

```
CONN.commit()
```

```
#!/usr/bin/env python3
#lib/testing/debug.py
```

```
from __init__ import CONN, CURSOR
from department import Department
```

```
import ipdb
```

```
Department.drop_table()
Department.create_table()
```

```
payroll = Department.create("Payroll", "Building A, 5th Floor")
print(payroll) # <Department 1: Payroll, Building A, 5th Floor>
```

```
hr = Department.create("Human Resources", "Building C, East Wing")
print(hr) # <Department 2: Human Resources, Building C, East Wing>
```

```
hr.name = 'HR'
hr.location = "Building F, 10th Floor"
hr.update()
print(hr) # <Department 2: HR, Building F, 10th Floor>
```

```
print("Delete Payroll")
payroll.delete() # delete from db table, object still exists in memory
print(payroll) # <Department 1: Payroll, Building A, 5th Floor>
```

```
ipdb.set_trace()
```

Resources

- [sqlite3 - DB-API 2.0 interface for SQLite databases - Python](https://docs.python.org/3/library/sqlite3.html)  (<https://docs.python.org/3/library/sqlite3.html>).

This tool needs to be loaded in a new browser window

Load Mapping A Python Class to a Database Table : Code-Along (CodeGrade) in a new window